

Nome: João Geiger Piza

Explicação sobre aumento de dados e deformações elásticas:

O aumento de dados é uma técnica que cria novos exemplos de treinamento a partir de dados originais para ajudar a inteligência artificial a aprender de forma mais abrangente, sem apenas memorizar informações. As deformações elásticas são um tipo específico dessa técnica usado principalmente em imagens, onde a figura é esticada ou distorcida, ensinando o sistema a reconhecer um objeto mesmo quando sua forma e tamanho variam naturalmente.

Descrição da implementação da U-net em PyTorch:

A U-Net foi implementada por meio da criação e definição das classes ConvBlock, UpConvBlock, Encoder, Decoder, Bottleneck e FinalConv, unindo todas na classe U-Net, que representa nosso modelo.

Descrição do Dataset utilizado:

O OralEpitheliumDB_Dataset é um conjunto de dados público desenvolvido para pesquisas em segmentação e classificação de imagens histopatológicas de displasia epitelial oral. O banco de dados contém 456 imagens obtidas de 30 línguas de camundongos, coradas por hematoxilina e eosina, abrangendo diferentes graus de displasia. Além das imagens originais, o conjunto disponibiliza máscaras de referência com os núcleos celulares anotados manualmente por especialistas e validados por um patologista.

Descrição dos experimentos com e sem aumento de dados:

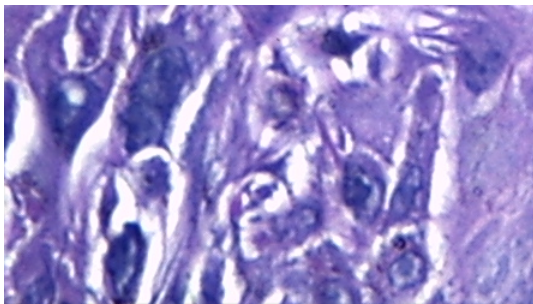
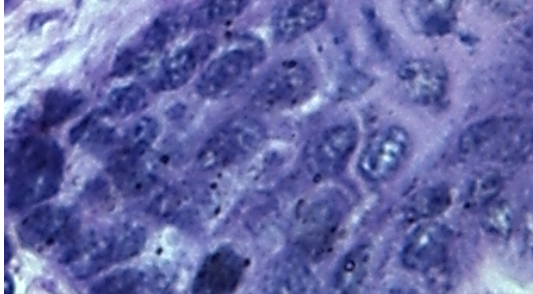
Ambos foram feitos usando a função de perda Binary Cross Entropy com Logit, o otimizador Adam, taxa de aprendizado de $1e-4$, 25 épocas e com tamanho de batch sendo 4.

Tabela com os resultados de DICE, acurácia de pixels e IoU:

Experimento	DICE	Acurácia de pixels	IoU	Observações
U-Net sem aumento de dados	77,05%	87,27%	62,90%	Nenhuma
U-Net com aumento de dados	77,62%	87,71%	63,73%	Aumentos usados: Horizontal flip, Vertical flip e shift scale rotate, todos com probabilidade de 20%.

Exemplos visuais contendo imagem original, máscara real e máscara prevista:

Imagens originais (healthy-02-roi5 e severe-17-roi1, respectivamente):



máscaras reais (healthy-02-roi5 e severe-17-roi1, respectivamente):



máscaras previstas pelo modelo sem aumento de dados (healthy-02-roi5 e severe-17-roi1, respectivamente):





máscaras previstas pelo modelo com aumento de dados (healthy-02-roi5 e severe-17-roi1, respectivamente):



Análise crítica dos resultados:

- 1) A U-Net, apesar de seu ótimo desempenho, ainda assim “alucina” núcleos inexistentes e ignora alguns que realmente estão lá, logo não consegue corretamente segmentar os núcleos.
- 2) O aumento de dados geométrico apresentou melhora em torno de 0,6% de todas as métricas quando comparado ao modelo que não fez uso desta técnica.
- 3) A métrica com maior diferença foi a IoU, com diferença de 0,63%.
- 4) Não, pois se seguirmos apenas esta métrica, acreditamos que o modelo é muito melhor do que é de fato.
- 5) Pois o IoU essencialmente “ignora” o acerto do fundo e foca nos acertos de marcações positivas.
- 6) A U-Net apresentou mais dificuldades ao lidar com núcleos agrupados, deixando de indicar alguns e até “alucinando” inexistentes próximos.
- 7) O modelo, no geral, apresentou mais falsos positivos.
- 8) As skip connections permitiram a transferência de detalhes das camadas iniciais direto para as camadas finais, ajudando estas.
- 9) Os resultados comprovam o que foi dito no artigo, de que é um bom modelo para segmentação de dados e que o aumento de dados é muito útil para obter bons resultados.
- 10) A FCN possui um fluxo contínuo de redução da imagem e tenta recuperar a resolução apenas no final, usando conexões simples. A U-Net tem uma estrutura simétrica em “U”, utilizando conexões diretas (skip connections) em todos os níveis para ligar as camadas iniciais diretamente às finais.

Comparação conceitual entre FCN e U-Net:

A FCN classifica os pixels de uma imagem, mas costuma perder detalhes estruturais ao reduzir a resolução dos dados. A U-Net evolui esse conceito usando uma arquitetura simétrica dividida em codificação e decodificação, utilizando conexões que transferem os detalhes de alta resolução das camadas iniciais direto para as camadas finais de reconstrução.

Código:

modelo.py:

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class ConvBlock(nn.Module):
    def __init__(
        self,
        in_channels,
        out_channels,
        kernel_size=3,
        stride=1,
        padding=1,
        batchnorm=True,
        dropout_rate=0.0,
    ):
        super(ConvBlock, self).__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(
                in_channels,
                out_channels,
                kernel_size=kernel_size,
                stride=stride,
                padding=padding,
            ),
            nn.BatchNorm2d(num_features=out_channels) if batchnorm else
nn.Identity(),
            nn.ReLU(inplace=True),
            nn.Dropout2d(p=dropout_rate),
            nn.Conv2d(
                out_channels,
                out_channels,
                kernel_size=kernel_size,
```

```

        stride=stride,
        padding=padding,
    ),
    nn.BatchNorm2d(num_features=out_channels) if batchnorm else
nn.Identity(),
    nn.ReLU(inplace=True),
    nn.Dropout2d(p=dropout_rate),
)

def forward(self, x):
    return self.conv(x)

class Encoder(nn.Module):
    def __init__(self, in_channels, batchnorm, dropout_rate=0.0):
        super(Encoder, self).__init__()
        self.conv_blocks = nn.ModuleList(
            [
                ConvBlock(
                    in_channels=in_channels,
                    out_channels=64,
                    batchnorm=batchnorm,
                    dropout_rate=dropout_rate,
                ),
                ConvBlock(
                    in_channels=64,
                    out_channels=128,
                    batchnorm=batchnorm,
                    dropout_rate=dropout_rate,
                ),
                ConvBlock(
                    in_channels=128,
                    out_channels=256,
                    batchnorm=batchnorm,
                    dropout_rate=dropout_rate,
                ),
                ConvBlock(
                    in_channels=256,
                    out_channels=512,
                    batchnorm=batchnorm,
                    dropout_rate=dropout_rate,
                ),
            ]
        )

```

```

        ]

    )

    self.max_pool = nn.MaxPool2d(kernel_size=2, stride=2)

def forward(self, x):
    skip_connections = []
    for conv_block in self.conv_blocks:
        x = conv_block(x)
        skip_connections.append(x)
        x = self.max_pool(x)
    return x, skip_connections

class Bottleneck(nn.Module):
    def __init__(self, in_channels=512, out_channels=1024,
batchnorm=True, dropout_rate=0.0):
        super(Bottleneck, self).__init__()
        self.bottleneck = ConvBlock(
            in_channels=in_channels,
            out_channels=out_channels,
            batchnorm=batchnorm,
            dropout_rate=dropout_rate
        )

    def forward(self, x):
        return self.bottleneck(x)

class UpConvBlock(nn.Module):
    def __init__(self, in_channels, out_channels, batchnorm=True,
dropout_rate=0.0):
        super(UpConvBlock, self).__init__()
        self.up = nn.ConvTranspose2d(in_channels, out_channels,
kernel_size=2, stride=2)

        self.conv_block = ConvBlock(
            in_channels=out_channels * 2,
            out_channels=out_channels,
            batchnorm=batchnorm,
            dropout_rate=dropout_rate
        )

    def forward(self, x, skip_x):

```

```

        x = self.up(x)

        diffY = skip_x.size()[2] - x.size()[2]
        diffX = skip_x.size()[3] - x.size()[3]
        x = F.pad(x, [diffX // 2, diffX - diffX // 2,
                      diffY // 2, diffY - diffY // 2])
        x = torch.cat([x, skip_x], dim=1)
        return self.conv_block(x)

class Decoder(nn.Module):
    def __init__(self, batchnorm=True, dropout_rate=0.0):
        super(Decoder, self).__init__()
        self.up1 = UpConvBlock(in_channels=1024, out_channels=512,
                                batchnorm=batchnorm, dropout_rate=dropout_rate)
        self.up2 = UpConvBlock(in_channels=512, out_channels=256,
                                batchnorm=batchnorm, dropout_rate=dropout_rate)
        self.up3 = UpConvBlock(in_channels=256, out_channels=128,
                                batchnorm=batchnorm, dropout_rate=dropout_rate)
        self.up4 = UpConvBlock(in_channels=128, out_channels=64,
                                batchnorm=batchnorm, dropout_rate=dropout_rate)

    def forward(self, x, skip_connections):
        x = self.up1(x, skip_connections[3])
        x = self.up2(x, skip_connections[2])
        x = self.up3(x, skip_connections[1])
        x = self.up4(x, skip_connections[0])
        return x

class FinalConv(nn.Module):
    def __init__(self, in_channels, num_classes):
        super(FinalConv, self).__init__()
        self.conv = nn.Conv2d(in_channels, num_classes, kernel_size=1)

    def forward(self, x):
        return self.conv(x)

class UNet(nn.Module):
    def __init__(self, in_channels=3, num_classes=1, dropout_rate=0.0,
                 batchnorm=True):
        super(UNet, self).__init__()

```

```

        self.encoder = Encoder(in_channels, batchnorm,
dropout_rate=dropout_rate)

        self.bottleneck = Bottleneck(512, 1024, batchnorm,
dropout_rate)

        self.decoder = Decoder(batchnorm, dropout_rate)

        self.final_conv = FinalConv(64, num_classes)

    def forward(self, x):

        output, skip_connections = self.encoder(x)

        output = self.bottleneck(output)

        output = self.decoder(output, skip_connections)

        output = self.final_conv(output)

        return output

```

configs.py:

```

import torch

import torch.nn as nn

from modelo import UNet

SEED = 42

F_LOSS = nn.BCEWithLogitsLoss()

OTIMIZADOR = torch.optim.Adam

TAXA_APRENDIZADO = 1e-4

NUM_EPOCAS = 25

TAM_BATCH = 4

DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

SUAVIZACAO = 1e-8

def pegar_modelo():

    modelo = UNet(in_channels=3, num_classes=1, dropout_rate=0.1,
batchnorm=True)

    return modelo

```

paths.py:

```

import os

DIRETORIO = os.path.dirname(os.path.abspath(__file__))

DATA = os.path.join(DIRETORIO, "data")

RAW = os.path.join(DATA, "raw")

GOLDEN = os.path.join(DATA, "golden")

MASCARAS = os.path.join(DATA, "mascaras")

```

```
MASCARAS_SEM = os.path.join(MASCARAS, "sem_aumento")
MASCARAS_COM = os.path.join(MASCARAS, "com_aumento")
```

prep_dados.py:

```
import os
import multiprocessing

import numpy as np
import albumentations as A
import torch

from torch import nn
from torch.utils.data import Dataset, DataLoader
from PIL import Image
from albumentations.pytorch import ToTensorV2
from sklearn.model_selection import train_test_split

from paths import RAW, GOLDEN
from configs import SEED, TAM_BATCH

class ConjuntoDadosTif(Dataset):
    def __init__(self, lista_arquivos, transformacoes):
        self.lista_arquivos = lista_arquivos
        self.transformacoes = transformacoes

    def __len__(self):
        return len(self.lista_arquivos)

    def __getitem__(self, indice):
        nome = self.lista_arquivos[indice]

        path_completo_imagem = os.path.join(RAW, f"{nome}.tif")
        path_completo_mascara = os.path.join(GOLDEN, f"{nome}.png")

        imagem =
np.array(Image.open(path_completo_imagem).convert("RGB"))
        mascara =
np.array(Image.open(path_completo_mascara).convert("L"))

        mascara_binaria = (mascara > 0).astype(np.int64)

        aumentado = self.transformacoes(image=imagem,
```

```

mask=mascara_binaria)

    imagem_transformada = aumentado['image']
    mascara_transformada = aumentado['mask'].long()

    return imagem_transformada, mascara_transformada

def pegar_loades(aumento: bool):
    transformacoes_sem_aumento = A.Compose([
        A.Normalize(mean=(0.485, 0.456, 0.406), std=(0.229, 0.224,
0.225)),
        ToTensorV2()
    ])

    transformacoes_com_aumento = A.Compose([
        A.HorizontalFlip(p=0.2),
        A.VerticalFlip(p=0.2),
        A.ShiftScaleRotate(shift_limit=0.05, scale_limit=0.1,
rotate_limit=45, p=0.2),
        A.Normalize(mean=(0.485, 0.456, 0.406), std=(0.229, 0.224,
0.225)),
        ToTensorV2()
    ])

    arquivos_totais = [
        os.path.splitext(arquivo)[0]
        for arquivo in os.listdir(RAW)
        if arquivo.endswith('.tif')
    ]

    arquivos_treino, arquivos_teste = train_test_split(
        arquivos_totais,
        test_size=0.2,
        random_state=SEED
    )

    if not aumento:
        dataset_treino_normal = ConjuntoDadosTif(arquivos_treino,
transformacoes_sem_aumento)
        dataset_teste_normal = ConjuntoDadosTif(arquivos_teste,
transformacoes_sem_aumento)
        carregador_treino = DataLoader(dataset_treino_normal,
batch_size=TAM_BATCH, shuffle=True, pin_memory=True)

```

```

        carregador_teste = DataLoader(dataset_teste_normal,
batch_size=TAM_BATCH, shuffle=False, pin_memory=True)
    else:
        dataset_treino_aumentado = ConjuntoDadosTif(arquivos_treino,
transformacoes_com_aumento)
        dataset_teste_aumentado = ConjuntoDadosTif(arquivos_teste,
transformacoes_sem_aumento)
        carregador_treino = DataLoader(dataset_treino_aumentado,
batch_size=TAM_BATCH, shuffle=True, pin_memory=True)
        carregador_teste = DataLoader(dataset_teste_aumentado,
batch_size=TAM_BATCH, shuffle=False, pin_memory=True)

    return carregador_treino, carregador_teste

```

metricas.py:

```

import torch

from configs import SUAVIZACAO

def calcular_dice(saidas, mascaras):
    previsoes = (saidas > 0).float().flatten()
    mascaras = mascaras.flatten()

    intersecao = (previsoes * mascaras).sum()
    soma = previsoes.sum() + mascaras.sum()

    dice = (2.0 * intersecao + SUAVIZACAO) / (soma + SUAVIZACAO)
    return dice.item()

def calcular_acuracia_pixel(saidas, mascaras):
    previsoes = (saidas > 0).float().flatten()
    mascaras = mascaras.flatten()

    corretos = (previsoes == mascaras).sum()
    total = mascaras.numel()

    acuracia = corretos / total
    return acuracia.item()

def calcular_iou(saidas, mascaras):
    previsoes = (saidas > 0).float().flatten()
    mascaras = mascaras.flatten()

```

```

intersecao = (previsoes * mascaras).sum()
uniao = previsoes.sum() + mascaras.sum() - intersecao

iou = (intersecao + SUAVIZACAO) / (uniao + SUAVIZACAO)
return iou.item()

```

treino.py:

```

import torch

import configs as cfg
from prep_dados import pegar_loades
from metricas import calcular_dice, calcular_acuracia_pixel,
calcular_iou

def treinar(aumento: bool):
    device = cfg.DEVICE
    print(f"Treinando com: {device}")
    modelo = cfg.pegar_modelo()
    modelo.to(device)
    otim = cfg.OTIMIZADOR(modelo.parameters(), lr=cfg.TAXA_APRENDIZADO)
    loader_treino, loader_teste = pegar_loades(aumento)

    for epoca in range(cfg.NUM_EPOCAS):
        modelo.train()
        perda_total_treino = 0.0

        for img, msc in loader_treino:
            img = img.to(device)
            msc = msc.unsqueeze(1).to(device).float()

            otim.zero_grad()

            saida = modelo(img)

            perda = cfg.F_LOSS(saida, msc)

            perda.backward()
            otim.step()
            perda_total_treino += perda.item()

        modelo.eval()

```

```

        perda_total_teste = 0.0
        dice_total = 0.0
        acuracia_total = 0.0
        iou_total = 0.0

    with torch.no_grad():
        for img, msc in loader_teste:
            img = img.to(device)
            msc = msc.unsqueeze(1).to(device).float()

            saida = modelo(img)

            perda = cfg.F_LOSS(saida, msc)
            perda_total_teste += perda.item()

            dice_total += calcular_dice(saida, msc)
            acuracia_total += calcular_acuracia_pixel(saida, msc)
            iou_total += calcular_iou(saida, msc)

    print(f'Epoca {epoca+1}:')
    print(f'- Perda no treino:
{perda_total_treino/len(loader_treino):.4f}')
    print(f'- Perda no teste:
{perda_total_teste/len(loader_teste):.4f}')
    print(f'- DICE: {dice_total/len(loader_teste):.4f}')
    print(f'- Pixel Accuracy:
{acuracia_total/len(loader_teste):.4f}')
    print(f'- IoU: {iou_total/len(loader_teste):.4f}')

    return modelo

```

main.py:

```

import os
import torch
import numpy as np
from PIL import Image
import albumentations as A
from albumentations.pytorch import ToTensorV2

import configs as cfg
from paths import RAW, MASCARAS_SEM, MASCARAS_COM
from treino import treinar

```

```

def gerar_mascaras(modelo, pasta_destino):
    os.makedirs(pasta_destino, exist_ok=True)

    device = cfg.DEVICE
    modelo.eval()

    transformacao_inferencia = A.Compose([
        A.Normalize(mean=(0.485, 0.456, 0.406), std=(0.229, 0.224,
0.225)),
        ToTensorV2()
    ])

    arquivos = [arquivo for arquivo in os.listdir(RAW) if
arquivo.endswith('.tif')]

    with torch.no_grad():
        for arquivo in arquivos:
            caminho_imagem = os.path.join(RAW, arquivo)

            imagem_pil = Image.open(caminho_imagem).convert("RGB")
            imagem_np = np.array(imagem_pil)

            tensor_imagem =
transformacao_inferencia(image=imagem_np)['image']
            tensor_imagem = tensor_imagem.unsqueeze(0).to(device)

            saidas = modelo(tensor_imagem)

            mascara_predita = (saidas > 0).squeeze().cpu().numpy()

            mascara_visivel = (mascara_predita * 255).astype(np.uint8)

            nome_puro = os.path.splitext(arquivo)[0]
            caminho_salvar = os.path.join(pasta_destino,
f"{nome_puro}.png")
            Image.fromarray(mascara_visivel).save(caminho_salvar)

if __name__ == '__main__':
    print("Iniciando treinamento SEM aumento de dados...")
    modelo_sem = treinar(False)

    print("\nGerando previsões para o modelo SEM aumento...")

```

```
gerar_mascaras(modelo_sem, MASCARAS_SEM)

print("\nIniciando treinamento COM aumento de dados...")
modelo_com = treinar(True)

print("Gerando previsões para o modelo COM aumento...")
gerar_mascaras(modelo_com, MASCARAS_COM)

print("Processo finalizado com sucesso!")
```